

Operatori in Python

Operatori in Python

I simboli con cui fare calcoli, confronti e operazioni logiche in Python.

Cosa sono gli operatori?

Un **operatore** è un simbolo che esegue un'operazione su uno o più valori. Li usi già ogni giorno: `+`, `-`, `*` sono operatori matematici. In Python ce ne sono molti altri.

Operatori aritmetici — per i calcoli

Funzionano come la matematica che conosci già, con qualche aggiunta:

Operatore	Significato	Esempio	Risultato
<code>+</code>	Addizione	<code>5 + 3</code>	<code>8</code>
<code>-</code>	Sottrazione	<code>5 - 3</code>	<code>2</code>
<code>*</code>	Moltiplicazione	<code>5 * 3</code>	<code>15</code>
<code>/</code>	Divisione (con decimali)	<code>5 / 2</code>	<code>2.5</code>
<code>//</code>	Divisione intera (senza decimali)	<code>5 // 2</code>	<code>2</code>
<code>%</code>	Resto della divisione	<code>5 % 2</code>	<code>1</code>
<code>**</code>	Potenza	<code>5 ** 2</code>	<code>25</code>

L'operatore `//` (doppio slash) divide e butta via i decimali. È come chiedere "quante volte 2 entra in 5?" — la risposta è 2, con 1 di resto.

L'operatore `%` (modulo) restituisce solo il resto. Torna utilissimo per sapere se un numero è pari o dispari: se `numero % 2` vale `0`, il numero è pari.

```

a = 10
b = 3
print(a + b)    # 13
print(a - b)    # 7
print(a * b)    # 30
print(a / b)    # 3.3333...
print(a // b)   # 3    (10 diviso 3 fa 3 con il resto)
print(a % b)    # 1    (il resto è 1)
print(a ** b)   # 1000 (10 alla potenza 3)

```

Operatori di confronto — per fare domande

Confrontano due valori e restituiscono sempre `True` o `False` :

Operatore	Significato	Esempio
<code>==</code>	È uguale a?	<code>5 == 5</code> → <code>True</code>
<code>!=</code>	È diverso da?	<code>5 != 3</code> → <code>True</code>
<code>></code>	È maggiore di?	<code>5 > 4</code> → <code>True</code>
<code><</code>	È minore di?	<code>5 < 3</code> → <code>False</code>
<code>>=</code>	È maggiore o uguale?	<code>5 >= 5</code> → <code>True</code>
<code><=</code>	È minore o uguale?	<code>5 <= 4</code> → <code>False</code>

```

x = 5
print(x == 5)  # True
print(x != 3)  # True
print(x > 4)   # True
print(x < 3)   # False

```

Operatori logici — per combinare condizioni

Servono a mettere insieme più condizioni:

Operatore	Significato
<code>and</code>	Vero solo se tutte le condizioni sono vere
<code>or</code>	Vero se almeno una condizione è vera
<code>not</code>	Inverte il valore (vero diventa falso e viceversa)

```
print(5 > 3 and 10 > 7) # True - entrambe vere
print(5 > 3 or 10 < 7)  # True - almeno una vera
print(not 5 > 3)         # False - 5 > 3 è True, not lo inverte
```

Operatori di assegnazione — scorciatoie per aggiornare variabili

Invece di scrivere `x = x + 3`, puoi usare la versione abbreviata `x += 3`. Significano la stessa cosa:

Abbreviazione	Equivalente completo
<code>x += 3</code>	<code>x = x + 3</code>
<code>x -= 3</code>	<code>x = x - 3</code>
<code>x *= 3</code>	<code>x = x * 3</code>
<code>x /= 3</code>	<code>x = x / 3</code>
<code>x //= 3</code>	<code>x = x // 3</code>
<code>x %= 3</code>	<code>x = x % 3</code>
<code>x **= 3</code>	<code>x = x ** 3</code>

```
punteggio = 100
punteggio += 50 # il giocatore ha guadagnato 50 punti
print(punteggio) # 150

punteggio -= 20 # penalità di 20 punti
print(punteggio) # 130
```

Operatori **in** e **not in** — per cercare elementi

Verificano se un valore è presente (o assente) in una lista, in una stringa o in qualsiasi altra sequenza:

```
frutta = ["mela", "banana", "ciliegia"]
print("mela" in frutta)      # True  - c'è la mela
print("uva" in frutta)      # False - non c'è l'uva
print("uva" not in frutta)   # True  - l'uva non c'è

testo = "Ciao, mondo!"
print("mondo" in testo)     # True
print("pizza" in testo)     # False
```

L'ordine delle operazioni

Come in matematica, Python esegue le operazioni in un ordine preciso. Le parentesi hanno sempre la precedenza massima:

```
print(2 + 3 * 4)    # 14 - prima la moltiplicazione, poi l'addizione
print((2 + 3) * 4)  # 20 - prima le parentesi
```

Se non sei sicuro dell'ordine, usa le parentesi: rendono il codice più chiaro e sicuro.